BigML: Coding

Alexander Lio

Env and Imports

Reproducibility and Setup

```
Seed: 42
Device: mps
```

Load SpuCoMNIST

Only the required arguments (classes, difficulty) and the train-split correlation strength are set, following the SpuCo quickstart. All other parameters are left at their defaults. `cm.get_cmap` patched because I am on a newer matplotlib version, did not want to reset venv.

```
48004 11996 10000
torch.Size([3, 28, 28]) 2
(0, 0) 10082
(0, 1) 11
(0, 2) 15
(0, 3) 10
(0, 4) 15
(1, 0) 13
(1, 1) 9623
(1, 2) 11
(1, 3) 8
(1, 4) 17
(2, 0) 13
(2, 1) 9
(2, 2) 8965
(2, 3) 14
(2, 4) 10
(3, 0) 14
(3, 1) 11
(3, 2) 11
(3, 3) 9698
(3, 4) 13
(4, 0) 12
(4, 1) 13
(4, 2) 13
(4, 3) 10
(4, 4) 9393
```

Visualize the data





Define ERM Model + Train

```
/Users/alexanderlio/Desktop/BigML-SpuCo/.venv/lib/python3.14/site-packages/torch/utils/data/dataloader.py:1102: UserWarning: 'pin_memory' argument is set as true but not supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
```

Evaluate the ERM Baseline

```
Evaluating group-wise accuracy:   4%|▍         | 1/25 [00:08<03:34,  8.93s/it]
Group (0, 0) Accuracy: 100.0
Evaluating group-wise accuracy:   8%|▊         | 2/25 [00:17<03:15,  8.49s/it]
Group (0, 1) Accuracy: 0.0
Evaluating group-wise accuracy:  12%|█▏        | 3/25 [00:25<03:03,  8.33s/it]
Group (0, 2) Accuracy: 0.0
Evaluating group-wise accuracy:  16%|█▌        | 4/25 [00:33<02:51,  8.15s/it]
Group (0, 3) Accuracy: 0.0
Evaluating group-wise accuracy:  20%|██        | 5/25 [00:41<02:43,  8.20s/it]
Group (0, 4) Accuracy: 0.0
Evaluating group-wise accuracy:  24%|██▍       | 6/25 [00:49<02:34,  8.16s/it]
Group (1, 0) Accuracy: 0.0
Evaluating group-wise accuracy:  28%|██▊       | 7/25 [00:57<02:25,  8.07s/it]
Group (1, 1) Accuracy: 100.0
Evaluating group-wise accuracy:  32%|███▏      | 8/25 [01:05<02:16,  8.03s/it]
Group (1, 2) Accuracy: 0.0
Evaluating group-wise accuracy:  36%|███▌      | 9/25 [01:13<02:07,  7.98s/it]
Group (1, 3) Accuracy: 0.0
Evaluating group-wise accuracy:  40%|████      | 10/25 [01:21<02:00,  8.03s/it]
Group (1, 4) Accuracy: 0.0
Evaluating group-wise accuracy:  44%|████▍     | 11/25 [01:29<01:51,  7.96s/it]
Group (2, 0) Accuracy: 0.0
Evaluating group-wise accuracy:  48%|████▊     | 12/25 [01:37<01:43,  7.95s/it]
Group (2, 1) Accuracy: 0.0
Evaluating group-wise accuracy:  52%|█████▏    | 13/25 [01:45<01:35,  7.96s/it]
Group (2, 2) Accuracy: 100.0
Evaluating group-wise accuracy:  56%|█████▌    | 14/25 [01:52<01:25,  7.82s/it]
Group (2, 3) Accuracy: 0.0
Evaluating group-wise accuracy:  60%|██████    | 15/25 [02:00<01:18,  7.82s/it]
Group (2, 4) Accuracy: 0.0
Evaluating group-wise accuracy:  64%|██████▍   | 16/25 [02:07<01:09,  7.73s/it]
Group (3, 0) Accuracy: 0.0
Evaluating group-wise accuracy:  68%|██████▊   | 17/25 [02:15<01:01,  7.63s/it]
Group (3, 1) Accuracy: 0.0
Evaluating group-wise accuracy:  72%|███████▏  | 18/25 [02:22<00:53,  7.63s/it]
Group (3, 2) Accuracy: 0.0
```

```
Evaluating group-wise accuracy:  76%|███████   | 19/25 [02:30<00:46,  7.75s/it]
Group (3, 3) Accuracy: 100.0
Evaluating group-wise accuracy:  80%|████████  | 20/25 [02:39<00:39,  7.98s/it]
Group (3, 4) Accuracy: 0.0
Evaluating group-wise accuracy:  84%|████████  | 21/25 [02:47<00:31,  7.97s/it]
Group (4, 0) Accuracy: 0.0
Evaluating group-wise accuracy:  88%|████████  | 22/25 [02:54<00:23,  7.82s/it]
Group (4, 1) Accuracy: 0.0
Evaluating group-wise accuracy:  92%|████████  | 23/25 [03:02<00:15,  7.82s/it]
Group (4, 2) Accuracy: 0.0
Evaluating group-wise accuracy:  96%|████████  | 24/25 [03:10<00:07,  7.75s/it]
Group (4, 3) Accuracy: 0.0
Evaluating group-wise accuracy: 100%|██████████| 25/25 [03:18<00:00,  7.93s/it]
Group (4, 4) Accuracy: 100.0
((0, 1), 0.0) 99.49379218398467
```

Extract ERM Features

```
torch.Size([48004, 84]) (48004,)
```

Cluster Examples (GEORGE group inference)

```
Clustering class-wise: 100%|██████████| 5/5 [00:15<00:00,  3.06s/it]
18 groups
{(2, 0): 8965, (2, 1): 46, (0, 0): 3577, (0, 1): 914, (0, 2): 558, (0, 3): 51, (0,
4): 3961, (0, 5): 1072, (4, 0): 9393, (4, 1): 48, (1, 0): 4245, (1, 1): 818, (1, 2):
3557, (1, 3): 1052, (3, 0): 3168, (3, 1): 196, (3, 2): 5636, (3, 3): 747}
```

Discovered Groups

	pseudo-group	size	color 0	color 1	color 2	color 3	color 4
0	(0, 0)	3577	3577	0	0	0	0
1	(0, 1)	914	914	0	0	0	0
2	(0, 2)	558	558	0	0	0	0
3	(0, 3)	51	0	11	15	10	15
4	(0, 4)	3961	3961	0	0	0	0
5	(0, 5)	1072	1072	0	0	0	0
6	(1, 0)	4245	0	4245	0	0	0
7	(1, 1)	818	0	818	0	0	0
8	(1, 2)	3557	13	3508	11	8	17
9	(1, 3)	1052	0	1052	0	0	0
10	(2, 0)	8965	0	0	8965	0	0
11	(2, 1)	46	13	9	0	14	10
12	(3, 0)	3168	0	0	2	3166	0
13	(3, 1)	196	0	0	5	191	0
14	(3, 2)	5636	14	11	4	5594	13
15	(3, 3)	747	0	0	0	747	0
16	(4, 0)	9393	0	0	0	0	9393
17	(4, 1)	48	12	13	13	10	0

Group-Balanced Retraining

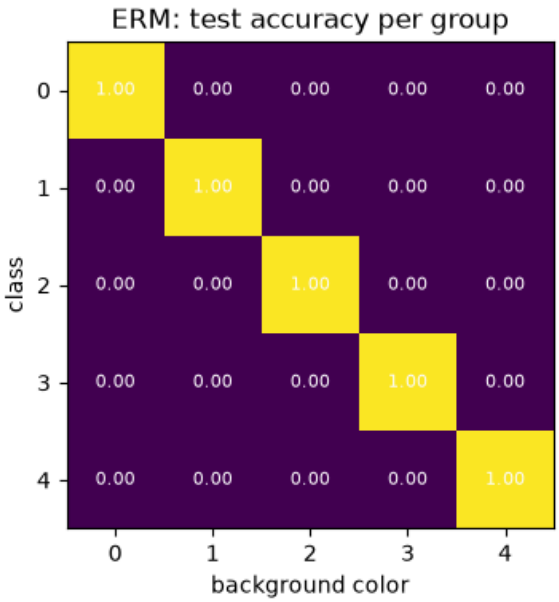
```
/Users/alexanderlio/Desktop/BigML-SpuCo/.venv/lib/python3.14/site-packages/torch/utils/data/dataloader.py:1102: UserWarning: 'pin_memory' argument is set as true but not supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
```

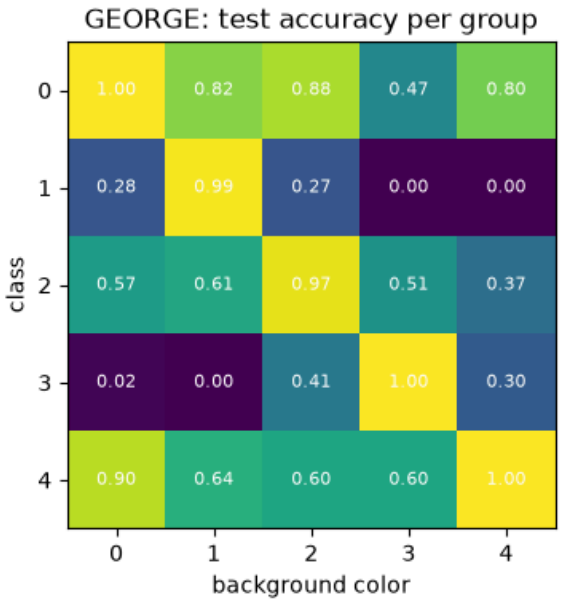
Evaluate the Retrained Model

```
Evaluating group-wise accuracy:   4%|▌         | 1/25 [00:07<03:03,  7.65s/it]
Group (0, 0) Accuracy: 99.76359338061465
Evaluating group-wise accuracy:   8%|▉         | 2/25 [00:15<02:55,  7.65s/it]
Group (0, 1) Accuracy: 82.03309692671395
Evaluating group-wise accuracy:  12%|█▎        | 3/25 [00:22<02:47,  7.64s/it]
Group (0, 2) Accuracy: 87.70685579196217
Evaluating group-wise accuracy:  16%|█▋        | 4/25 [00:30<02:41,  7.67s/it]
Group (0, 3) Accuracy: 46.808510638297875
Evaluating group-wise accuracy:  20%|██        | 5/25 [00:38<02:34,  7.73s/it]
Group (0, 4) Accuracy: 79.66903073286052
```

```
Evaluating group-wise accuracy:  24%|██        | 6/25 [00:45<02:25,  7.65s/it]
Group (1, 0) Accuracy: 27.87286063569682
Evaluating group-wise accuracy:  28%|███       | 7/25 [00:53<02:17,  7.61s/it]
Group (1, 1) Accuracy: 98.53300733496333
Evaluating group-wise accuracy:  32%|███       | 8/25 [01:01<02:09,  7.65s/it]
Group (1, 2) Accuracy: 27.45098039215686
Evaluating group-wise accuracy:  36%|████      | 9/25 [01:08<02:02,  7.63s/it]
Group (1, 3) Accuracy: 0.0
Evaluating group-wise accuracy:  40%|████      | 10/25 [01:16<01:54,  7.64s/it]
Group (1, 4) Accuracy: 0.0
Evaluating group-wise accuracy:  44%|█████     | 11/25 [01:24<01:47,  7.66s/it]
Group (2, 0) Accuracy: 56.53333333333333
Evaluating group-wise accuracy:  48%|█████     | 12/25 [01:31<01:39,  7.67s/it]
Group (2, 1) Accuracy: 60.8
Evaluating group-wise accuracy:  52%|██████    | 13/25 [01:39<01:33,  7.80s/it]
Group (2, 2) Accuracy: 96.8
Evaluating group-wise accuracy:  56%|██████    | 14/25 [01:47<01:25,  7.81s/it]
Group (2, 3) Accuracy: 51.46666666666667
Evaluating group-wise accuracy:  60%|███████   | 15/25 [01:55<01:17,  7.77s/it]
Group (2, 4) Accuracy: 36.63101604278075
Evaluating group-wise accuracy:  64%|███████   | 16/25 [02:03<01:10,  7.79s/it]
Group (3, 0) Accuracy: 1.5075376884422111
Evaluating group-wise accuracy:  68%|████████  | 17/25 [02:10<01:01,  7.74s/it]
Group (3, 1) Accuracy: 0.0
Evaluating group-wise accuracy:  72%|████████  | 18/25 [02:18<00:54,  7.74s/it]
Group (3, 2) Accuracy: 41.30982367758186
Evaluating group-wise accuracy:  76%|█████████ | 19/25 [02:26<00:46,  7.78s/it]
Group (3, 3) Accuracy: 99.74811083123426
Evaluating group-wise accuracy:  80%|█████████ | 20/25 [02:34<00:38,  7.71s/it]
Group (3, 4) Accuracy: 29.974811083123427
Evaluating group-wise accuracy:  84%|██████████| 21/25 [02:41<00:30,  7.70s/it]
Group (4, 0) Accuracy: 90.17632241813602
Evaluating group-wise accuracy:  88%|██████████| 22/25 [02:49<00:23,  7.74s/it]
Group (4, 1) Accuracy: 63.727959697733
Evaluating group-wise accuracy:  92%|██████████| 23/25 [02:57<00:15,  7.67s/it]
Group (4, 2) Accuracy: 59.949622166246854
Evaluating group-wise accuracy:  96%|██████████| 24/25 [03:04<00:07,  7.65s/it]
Group (4, 3) Accuracy: 60.101010101010104
Evaluating group-wise accuracy: 100%|██████████| 25/25 [03:12<00:00,  7.72s/it]
Group (4, 4) Accuracy: 99.74747474747475
((1, 3), 0.0) 98.68276352238568
```

Compare ERM vs. GEORGE Retraining





{'ERM': (np.float64(0.2), np.float64(0.0)), 'GEORGE': (np.float64(0.559), np.float64(0.452))}